CTU

CZECH TECHNICAL UNIVERSITY IN PRAGUE

F3

Faculty of Electrical Engineering
Department of Computer Science

Bachelor's Thesis

Web-based application for managing private bank accounts

Yelyzaveta Taranova
Software Engineering and Technology

Leden 2013, srpen 2020
Supervisor: Ondřej Vondrouš

Acknowledgement / Declaration

Chtěl bych poděkovat své manželce Ludmile za podporu nejen finanční. Díky tomu mohu na svém pracovišti dělat, co mě baví, a nejsem stresován výplatní páskou.

I declare that I have prepared the submitted thesis independently and that I have cited all the information sources used in accordance with the Methodological Guideline on the observance of ethical principles in the preparation of university final theses.

In Prague 13. 13. 2013 ???

.....

Abstrakt / Abstract

Tento projekt se zaměřuje na návrh a vývoj webové aplikace pro správu osobních bankovních účtů prostřednictvím agregace finančních údajů z více českých bank na jednom místě. Jeho hlavním cílem je poskytnout uživatelům jednotný, přehledný a uživatelsky přívětivý náhled na jejich finance, aniž by museli přepínat mezi různými systémy internetového bankovnictví.

Pro určení struktury a funkčnosti aplikace projekt začíná analýzou stávajících platforem pro správu osobních financí. Tato analýza identifikuje běžné funkce, silné stránky a omezení současných řešení a slouží jako základ pro návrh klíčových funkcí navrhované aplikace.

Klíčová slova: Bankovní aplikace, osobní bankovnictví, centralizace financí, webová aplikace, Spring Boot Framework, REST API, Java, službově orientovaná architektura.

This project focuses on the design and development of a web application for managing personal bank accounts by aggregating financial data from multiple Czech banks in one place. Its primary purpose is to provide users with a unified, clear, and user-friendly overview of their finances without requiring them to switch between different online banking systems.

To define the structure and functionality of the application, the project begins with an analysis of existing personal finance platforms. This analysis identifies common features, strengths, and limitations of current solutions and serves as the basis for designing the core functions of the proposed application.

Keywords: Banking application, personal banking, finance centralization, web application, Spring Boot Framework, REST API, Java, service-oriented architecture.

Contents /

1 Introduction **1**
1.1 Motivation 1
1.2 Objectives 1

2 Existing Solutions Overview **3**
2.1 PocketGuard 3
2.2 Personal Capital (Empower) . . . 3
2.3 Quicken Simplifi 3
2.4 Monarch Money 3
2.5 Key Features in Current Finance Apps 4

3 Business Analysis **5**
3.1 Functional Requirements 5
3.1.1 Access to Public Pages . . . 5
3.1.2 Registration and Login . . . 5
3.1.3 User Account Management . 6
3.1.4 Bank Integration 6
3.1.5 Financial Data Retrieval . . 6
3.1.6 Dashboard and Visualization 7
3.1.7 Transaction Categorization . 7
3.1.8 Budgeting 7
3.1.9 User Management 7
3.1.10 System Monitoring 8
3.1.11 Public Content 8
3.1.12 Fictive Bank Management . . 8
3.1.13 Fictive Account Management 8
3.2 Non-Functional Requirements . . 9
3.3 Use cases 9
3.3.1 Anonymous User 9
3.3.2 Logged-in User 9
3.3.3 Administrator 10
3.4 Process Analysis 10
3.4.1 Connecting a Bank Account 11

4 Technology Analysis **12**
4.1 Backend Development 12
4.1.1 Framework 12
4.1.2 Programming Language . . 12
4.1.3 Database 12
4.1.4 Caching 12
4.1.5 Resilience 13
4.1.6 Messaging Configuration . 13
4.1.7 Conclusion 13
4.2 Frontend Development 13
4.2.1 Framework 13

5 System Design **15**
5.1 Architecture 15
5.1.1 Architectural Style 15
5.1.2 Architecture Diagram . . . 15
5.1.3 Internal Architecture . . . 16
5.1.4 Component Diagram 17
5.2 Data design 19
5.2.1 Database Model 19
5.2.2 Domain Class Model 22
5.3 External API Overview 25
5.3.1 Conclusion 26

6 Conclusion **27**

References **28**

A Zadání práce **29**

B Zkratky a symboly **30**
B.1 Zkratky 30
B.2 Symboly 30
B.3 Soubory, které jsou součástí CTUstyle3 31

Tables / Figures

3.1 Functional Requirements — Access to Public Pages 5
3.2 Functional Requirements — Registration and Login 5
3.3 Functional Requirements — User Account Management 6
3.4 Functional Requirements — Bank Account Integration 6
3.5 Functional Requirements — Financial Data Retrieval 6
3.6 Functional Requirements — Dashboard and Visualization 7
3.7 Functional Requirements — Transaction Categorization 7
3.8 Functional Requirements — Budgeting 7
3.9 Functional Requirements — User Management 7
3.10 Functional Requirements — System monitoring 8
3.11 Functional Requirements — Public Content 8
3.12 Functional Requirements — Fictive Bank Management 8
3.13 Functional Requirements — Fictive Account Management 8
3.14 Non-functional Requirements 9
4.1 Selected technologies for application backend development 13
3.1 Use Cases — Anonymous User .. 9
?? Use Cases — Logged-in User .. ??
3.3 Use Cases — Administrator ... 10
3.4 Process Diagram — Connecting a Bank Account 11
5.1 Application Architecture 16
5.2 Internal Architecture 17
5.3 Component Diagram 18
5.4 Party Schema 20
5.5 Authorized Client Schema 20
5.6 Bank Schema 21
5.7 Product Schema 21
5.8 Categorization Schema 22
5.9 Domain Class Model - Bank Service 23
5.10 Domain Class Model - Product Service 24
5.11 Domain Class Model - Categorization Service 25

Chapter 1
Introduction

Managing personal finances has become increasingly complex as people maintain multiple bank accounts, credit cards, and financial services across different institutions. While Czech banks provide secure online banking platforms, users often struggle with fragmented financial information spread across several systems.

The rise of PSD2 and open banking standards in the European Union enables third-party applications to securely access financial data from banks with the user's consent. This creates an opportunity for a unified personal finance tool that consolidates a user's accounts in one place and presents their financial situation clearly and intuitively.

This application leverages these modern banking APIs to provide a single, user-friendly web interface where users can view their balances, track transactions, monitor spending patterns, and better understand their financial habits. The app is intended for personal use and focuses on security, privacy, and clarity while avoiding any high-risk operations like payments or account modifications.

1.1 Motivation

Although Czech banks support PSD2 APIs, each bank's online environment remains separate. Users commonly switch between several banking apps — such as Česká spořitelna, ČSOB, Komerční banka, Air Bank, or Moneta Money Bank — to get a full picture of their finances. This fragmentation makes it difficult to track spending, analyze financial behavior, or understand overall liquidity.

Existing international financial tools often do not support Czech banking APIs or require third-party aggregators, which may raise privacy concerns or lack localized features.

This project is motivated by the need for:

- **Centralization** — keeping all accounts and transactions in one place.
- **Transparency** — understanding personal cash flow across all banks.
- **Control and privacy** — processing data securely and exclusively for the user.
- **Simplicity** — avoiding the complexity of full-scale fintech products and focusing instead on a private, personal tool.

1.2 Objectives

The main objective of this application is to provide a secure and intuitive web tool that aggregates financial data from multiple Czech banks and offers meaningful insights into the user's personal finances. Specifically, the project aims to:

1. Create a unified view of personal finances

   The application aims to provide users with a single place where they can see their financial situation across multiple Czech banks. Instead of switching between

different online banking portals, users should gain a complete overview from one interface.

2. Simplify the understanding of financial behavior

The goal is to help users better understand how they spend and manage money. The app will provide tools that highlight patterns, categorize expenses, and show meaningful trends over time – making financial decision-making easier and more informed.

3. Leverage open banking to improve accessibility

By using PSD2-compliant APIs offered by Czech banks, the application intends to demonstrate how open banking technology can be used to make financial data more accessible to people while maintaining security and transparency.

4. Analyze existing personal finance platforms and identify best practices

An important objective of the project is to study current personal finance applications — both global and regional — to understand their features, design patterns, strengths, and limitations. This analysis helps identify which functionalities are most valuable to users, what approaches are commonly used in the industry, and where opportunities exist to create a simpler, localized, or more user-friendly solution. The findings from this analysis directly inform the functional design of the proposed application.

Chapter 2
Existing Solutions Overview

Managing personal bank accounts involves planning and budgeting. It includes setting financial goals (such as saving or debt repayment), tracking income and expenses, and monitoring account balances and net worth. Modern personal finance tools connect to bank accounts to automatically import transactions, categorize spending, and provide insights (budgets, reports, alerts) to help users stay on top of their finances.

2.1 PocketGuard

This app offers an all-in-one overview of personal finances. It can track net worth and transactions by connecting to bank accounts. Its budgeting tools are flexible, with customizable categories and "rollover" budgets. PocketGuard also manages recurring incomes and bills in one place, plans debt payoff schedules, and provides spending insights and goal tracking. In effect, PocketGuard functions like an account aggregator and budgeter combined, with added tools for debt and goal planning.

2.2 Personal Capital (Empower)

Personal Capital is a tool that combines standard budgeting features with advanced investment tracking. It aggregates all financial accounts (bank, credit, brokerage, retirement, etc.) into a unified dashboard. Users receive cash flow and spending reports, asset allocation charts, investment performance, and retirement planning tools. Personal Capital excels at helping users track net worth and make investment decisions, whereas its budgeting features are solid but not as detailed as dedicated budgeting apps.

2.3 Quicken Simplifi

Quicken (a legacy personal finance company) offers the Simplifi app for modern budgeting. Simplifi connects to bank accounts, credit cards, investments, and even tracks property values. It provides real-time updates of transactions, allowing users to see balances and spending trends. Users can create custom budgets, set savings goals, plan debt payoff, and get alerts for upcoming bills and cash-flow changes. Simplifi also tracks subscriptions and other spending categories through personalized watchlists. In effect, it merges account aggregation, budgeting, goal tracking, and notification features in one app.

2.4 Monarch Money

Monarch is a finance app that emphasizes collaboration (couples/families). It links all account types, so users always know their net worth, day-to-day spending, and

cash flow. Monarch automatically detects recurring bills and subscriptions (streaming, memberships, etc.), helping users identify forgotten charges. It offers customizable budgets, detailed visual reports (charts of spending by category, income over time, net worth trends), and goal setting for various savings targets.

2.5 Key Features in Current Finance Apps

Based on the above analysis, popular personal finance tools commonly include the following features:

- **Account aggregation** — linking multiple bank, credit, investment, and loan accounts to show a unified financial picture (balances, transactions, net worth). Most apps automatically download transactions and update in real time.
- **Expense tracking and categorization** — automatic import of transactions with categorization (e.g., "groceries," "utilities") and the ability to edit or tag entries. This powers spending reports and budget tracking.
- **Budgeting tools** — custom budget creation by category, often monthly. Some apps use zero-based or envelope budgeting, while others offer flexible budget goals. Budgets typically include rollovers or adjustments between months. Alerts or progress bars show how actual spending compares to the budget.
- **Financial goal tracking** — features to set and monitor savings goals (e.g., emergency fund, vacation) and debt payoff plans. Users can automate savings or define targets for paying off credit cards.
- **Bill and subscription management** — detection and management of recurring payments. Apps find recurring bills/subscriptions and allow cancellation of unwanted services. Bill reminders or negotiation services may also be included.
- **Reporting and visualization** — charts and graphs summarizing finances (spending by category, cash flow over time, asset allocation, net worth history). Many apps provide dashboards or reports that help users understand spending trends and financial health.
- **Alerts and notifications** — alerts for overspending, low balances, upcoming bills, or unusual account activity. Notifications help users stay on track with budgets and avoid fees or missed payments.
- **Security and sync** — bank-level security (encryption, multi-factor authentication) and cloud sync across devices. All major apps emphasize data security and availability on web/mobile platforms.

Chapter 3
Business Analysis

This chapter defines the business context of the application and translates user needs into structured functional and non-functional requirements. It identifies the target audience, describes expected system behavior, and outlines key business processes supported by the application. The goal of this analysis is to ensure that the proposed solution aligns with real user needs while remaining technically feasible and compliant with regulatory constraints.

3.1 Functional Requirements

This section specifies the functional requirements of the application, defining what the system shall do from the user's perspective. The requirements are grouped by functional areas and cover all major interactions, including authentication, bank integration, financial data retrieval, visualization, budgeting, administration, and management of fictive banks and accounts.

3.1.1 Access to Public Pages

ID	Title	Description
FR1	Displaying the landing page	The system shall allow anonymous users to view the landing page.
FR2	Displaying application description	The system shall display a general description of the application and its purpose.
FR3	Displaying application features	The system shall show information about features, security, and supported banks.

Table 3.1. Functional Requirements — Access to Public Pages

3.1.2 Registration and Login

ID	Title	Description
FR4	Registration	The system shall allow anonymous users to create an account via the registration form.
FR5	Login	The system shall support login using email and password.
FR6	Reset password	The system shall allow anonymous users to request a password reset.

Table 3.2. Functional Requirements — Registration and Login

3.1.3 User Account Management

ID	Title	Description
FR7	Profile edition	The system shall allow users to view and edit their profile details.
FR8	Password change	The system shall allow users to change their password.
FR9	Account deletion	The system shall allow users to delete their account and all associated data.

Table 3.3. Functional Requirements — User Account Management

3.1.4 Bank Integration

ID	Title	Description
FR10	Linking a bank	The system shall allow users to link a bank via the PSD2 authorization flow.
FR11	Access tokens storage	The system shall securely store access tokens required for retrieving data.
FR12	Disconnection from a bank	The system shall allow users to disconnect any previously linked bank.

Table 3.4. Functional Requirements — Bank Account Integration

3.1.5 Financial Data Retrieval

ID	Title	Description
FR13	Account details and balances retrieval	The system shall retrieve account details and balances from connected banks.
FR14	Transaction history retrieval	The system shall retrieve the transaction history for each connected account.
FR15	Financial data caching	The system shall cache financial data periodically.

Table 3.5. Functional Requirements — Financial Data Retrieval

3.1.6 Dashboard and Visualization

ID	Title	Description
FR16	Displaying a dashboard	The system shall display a consolidated dashboard containing all linked accounts.
FR17	Displaying statistics	The system shall present charts and graphs of spending, income, and trends.
FR18	Displaying transactions	The system shall show categorized transactions in a clear list format.

Table 3.6. Functional Requirements — Dashboard and Visualization

3.1.7 Transaction Categorization

ID	Title	Description
FR19	Transactions categorization	The system shall automatically categorize transactions using predefined rules.

Table 3.7. Functional Requirements — Transaction Categorization

3.1.8 Budgeting

ID	Title	Description
FR20	Monthly budgets	The system shall allow users to set monthly budgets for categories.
FR21	Budget warnings	The system shall display warnings or highlights when a budget is exceeded.

Table 3.8. Functional Requirements — Budgeting

3.1.9 User Management

ID	Title	Description
FR22	Displaying a list of registered users	The system shall allow administrators to view a list of registered users.
FR23	User deactivation	The system shall allow administrators to deactivate or remove users.
FR24	Unlock account	The system shall allow administrators to reset user accounts (e.g., unlock accounts).

Table 3.9. Functional Requirements — User Management

3.1.10 System Monitoring

ID	Title	Description
FR25	System logs	The system shall provide logs for authentication attempts, API errors, and system faults.

Table 3.10. Functional Requirements — System monitoring

3.1.11 Public Content

ID	Title	Description
FR26	Public content update	The system shall allow administrators to update public content (e.g., landing page text, FAQ).

Table 3.11. Functional Requirements — Public Content

3.1.12 Fictive Bank Management

ID	Title	Description
FR27	Creation of fictive bank	The system shall allow users to create a fictive (custom) bank that is not part of the predefined supported banks list.
FR28	Deletion of fictive bank	The system shall allow users to delete a fictive (custom) bank.
FR29	Edition of fictive bank	The system shall allow users to edit a fictive (custom) bank.

Table 3.12. Functional Requirements — Fictive Bank Management

3.1.13 Fictive Account Management

ID	Title	Description
FR30	Creation of fictive account	The system shall allow users to create one or more fictive bank accounts associated with a fictive bank.
FR31	Deletion of fictive account	The system shall allow users to delete a fictive account.
FR32	Edition of fictive account	The system shall allow users to edit a fictive account.

Table 3.13. Functional Requirements — Fictive Account Management

3.2 Non-Functional Requirements

ID	Title	Description
NFR1	Security	The application must use secure storage and encrypted communication.
NFR2	Responsive UI	UI must be responsive and usable across devices.
NFR3	Resilient system	The system must handle API failures gracefully.
NFR4	PSD2	Data processing must respect PSD2 and GDPR rules.
NFR5	Application performance	The application should load the dashboard within 10 seconds.

Table 3.14. Non-functional Requirements

3.3 Use cases

This section presents the main use cases of the system, describing how different types of users interact with the application. Use cases are divided according to user roles and illustrate the scope of functionality available to each role. The diagrams provide a high-level overview of system behavior and support the understanding of functional requirements.

3.3.1 Anonymous User

Anonymous users are visitors who access the application without creating an account or logging in.

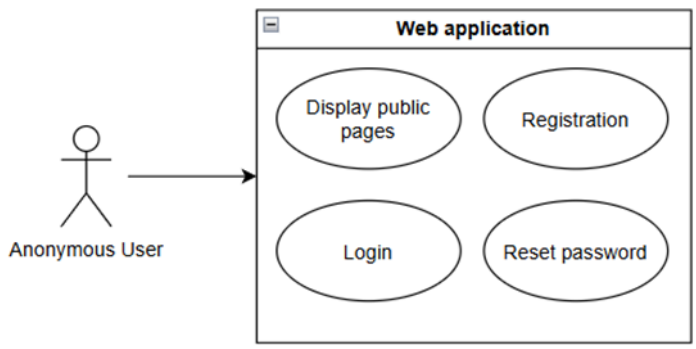

Figure 3.1. Use Cases — Anonymous User

3.3.2 Logged-in User

A logged-in user has access to the core features of the application, including connecting banking accounts and viewing financial data.

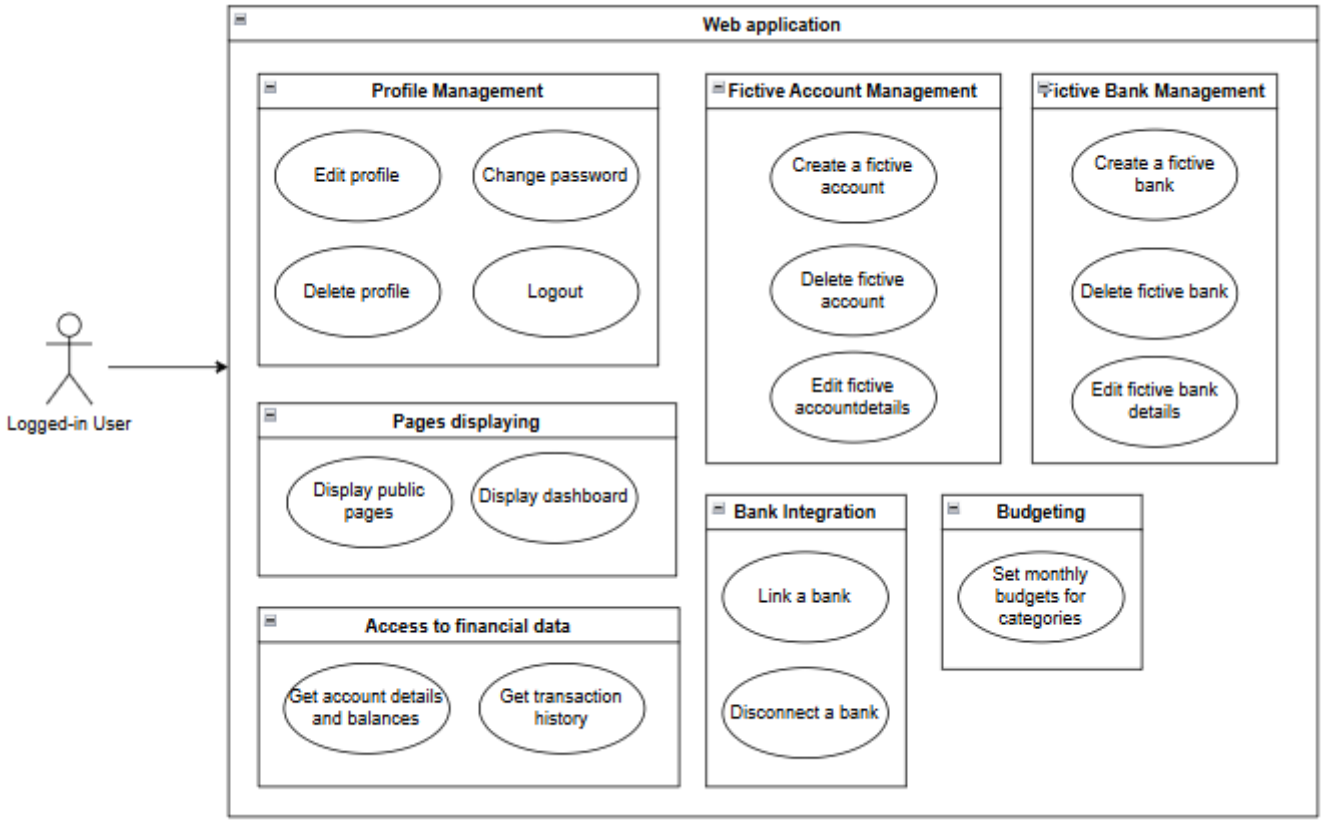

Figure 3.2. Use Cases — Logged-in User

3.3.3 Administrator

The administrator manages the system, users, and operational aspects. Admins do not have access to users' financial data.

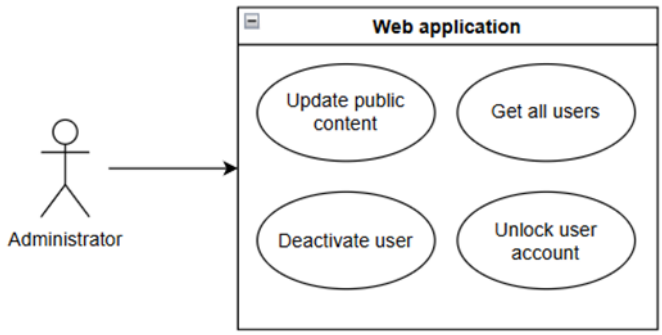

Figure 3.3. Use Cases — Administrator

3.4 Process Analysis

Process analysis describes the key workflows supported by the application and explains how individual components interact during typical operations. These processes focus on user interactions with external banking systems, internal data processing, and system responses. The aim is to clarify how functional requirements are realized in practice and to identify critical integration and security steps.

3.4.1 Connecting a Bank Account

The process of connecting a bank account enables the user to grant the application access to their financial data through an external banking API.

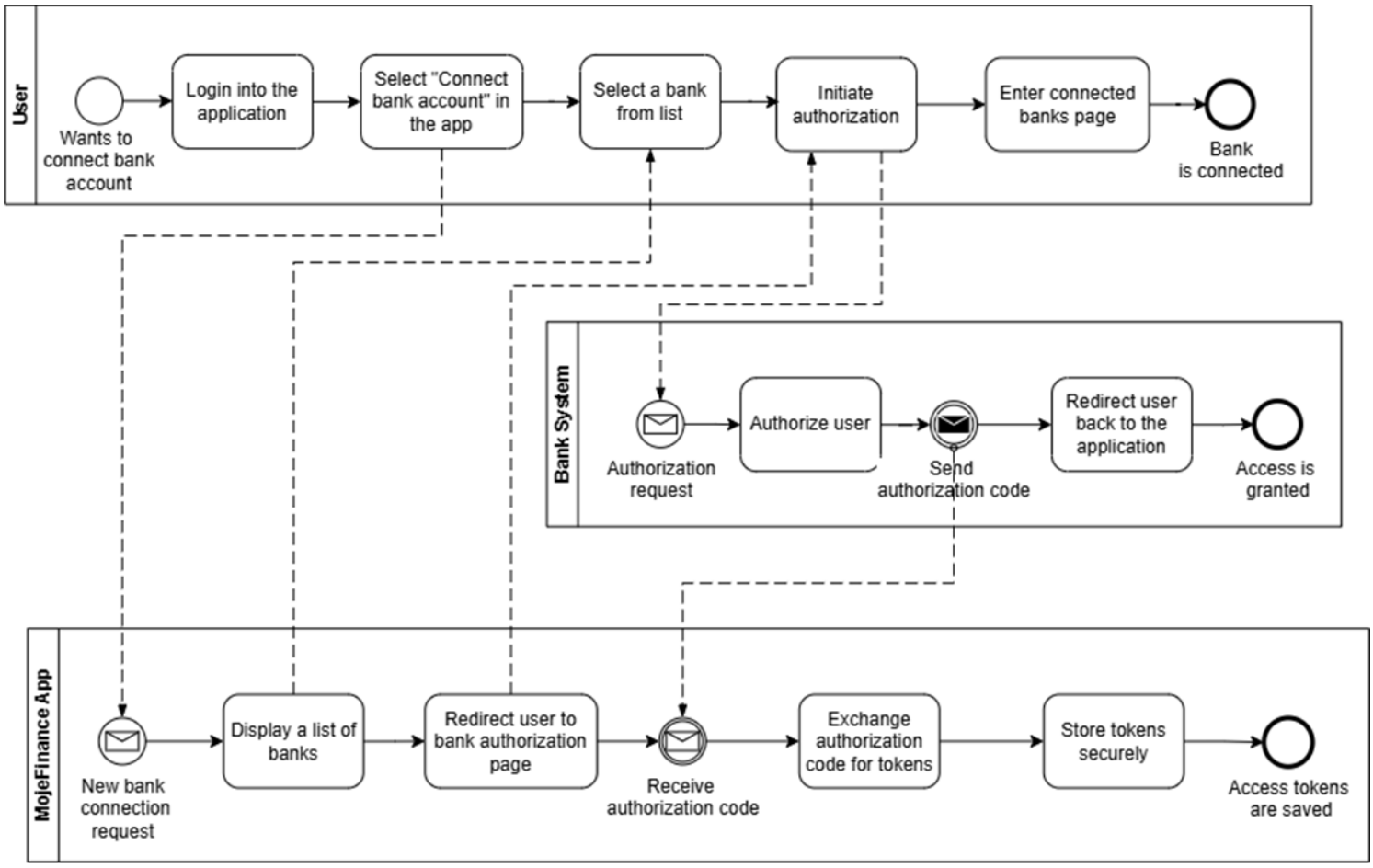

Figure 3.4. Process Diagram — Connecting a Bank Account

The process begins when a logged-in user selects a bank from the list of supported institutions within the application. After initiating the connection, the backend redirects the user to the selected bank's authentication page. At this stage, the user authenticates directly with their bank using the bank's own security mechanisms, such as two-factor authentication or biometric verification. The application itself never accesses or stores the user's banking credentials.

Once authentication is completed, the bank presents a consent screen where the user explicitly approves access to specific data scopes, typically including account information, balances, and transaction history. After consent is granted, the bank redirects the user back to the application along with an authorization code.

The backend service then exchanges this authorization code for an access token using a secure OAuth2 communication channel. The obtained tokens are securely stored in encrypted form and associated with the user's bank connection. The application uses these tokens to retrieve account and transaction data via the bank's API.

Following successful authorization, the system performs an initial synchronization, during which it retrieves the list of available accounts and their associated data. Retrieved accounts are normalized into the internal domain model and cached. Accounts become visible to the user in the application dashboard.

Chapter 4
Technology Analysis

This chapter analyzes the technologies selected for the development of the web application. The goal is to justify the chosen technology stack based on performance, maintainability, scalability, integration capabilities, and long-term sustainability.

4.1 Backend Development

4.1.1 Framework

The Boot framework offers standardized approaches to dependency injection, configuration management, service orchestration, and modular application design. A key characteristic of Spring Boot is its emphasis on scalability, maintainability, and integration capabilities. It supports complex architectures, microservices, advanced security mechanisms, observability tools, and strong ecosystem compatibility. This framework also promotes best practices such as layered architecture, convention over configuration, and dependency management, which contribute to robust and long-lasting software systems.

4.1.2 Programming Language

Java 21 was chosen as the primary programming language for the backend application. Java is an excellent choice for financial applications due to its strong security, high performance, and reliable cross-platform operation. Its seamless integration with the Spring Boot framework further enhances its suitability for building modern, robust banking systems.

4.1.3 Database

PostgreSQL was selected as the primary relational database management system. It is an open-source, enterprise-grade database known for reliability, extensibility, and advanced SQL compliance. The application requires structured storage for users, fictive accounts, and bank connections. PostgreSQL provides the robustness and consistency needed to handle sensitive financial records while enabling efficient querying for analytics.

4.1.4 Caching

Redis is used as an in-memory data store that enables fast access to frequently used information. Because bank APIs often impose request limits and may respond slowly during peak periods, caching selected data in Redis significantly improves responsiveness and reduces the load on both the database and external services. Temporary values such as session details, authorization tokens, or recently fetched account summaries can be stored with defined expiration times, ensuring that the system remains efficient while avoiding unnecessary external calls.

4.1.5 Resilience

In addition to caching, the application incorporates resilience mechanisms using **Resilience4j**. Since the system depends on multiple third-party banking APIs, communication reliability is a critical requirement. Resilience4j provides essential fault-tolerance patterns such as circuit breakers, retries, timeouts, and rate limiting. These mechanisms protect the application from failures caused by unavailable bank services, network latency, or unexpected response delays. By isolating faulty calls and preventing cascading failures, Resilience4j ensures that the application remains stable even when one of the external providers becomes temporarily unreachable.

4.1.6 Messaging Configuration

Feign Client is selected for communication with external APIs, it will provide a clear mechanism for synchronous data exchange with external services. It is a declarative HTTP client that simplifies the integration of REST services, especially when working with multiple banks. The application must integrate with several bank APIs, each having different endpoints, schemas, and authentication flows. Feign provides a clean and modular approach to defining these clients, making the codebase easier to maintain and extend.

4.1.7 Conclusion

The selected backend technologies form a robust and scalable foundation for a personal finance application. Java 21 and Spring Boot provide stability, security, and long-term maintainability. PostgreSQL ensures reliable and consistent data persistence, while Redis and Resilience4j improve performance and fault tolerance when communicating with external banking APIs. Feign Client simplifies API integration. Together, these technologies support a service-oriented architecture that is well suited for future extension and increased system complexity.

Selected technologies are summarized in the following table.

Category	Technology
Framework	Spring Boot
Programming Language	Java 21
Database	PostgreSQL
Caching	Redis
Resilience	Resilience4j
Messaging Configuration	Feign Client

Table 4.1. Selected technologies for application backend development

4.2 Frontend Development

4.2.1 Framework

React is a JavaScript library for building user interfaces, created and maintained by Meta (Facebook). It is currently one of the most popular and widely adopted frontend technologies worldwide. React organizes the interface into reusable components (buttons, forms, graphs, cards, charts). This is ideal for finance dashboards, which require

modular widgets. Additionally, React uses a virtual DOM, which makes UI updates fast when data changes frequently. Moreover, it has a massive ecosystem of libraries for routing, state management, charts, authentication, and UI styling, which dramatically reduces implementation effort.

Chapter 5
System Design

This chapter describes the overall system structure, architectural decisions, and internal organization of the application. It explains how individual services are arranged, how data flows through the system, and how external banking APIs are integrated. The design focuses on modularity, scalability, and maintainability while respecting security and regulatory requirements.

5.1 Architecture

This chapter presents the architectural design of the application, serving as the foundational blueprint for the entire system. Establishing a robust and well-defined architecture is crucial for ensuring the application's scalability, maintainability, and security — especially given the sensitive nature of financial data and the requirement to integrate with multiple third-party banking APIs securely. The following sections detail the chosen architectural style, the high-level system components, the internal structuring of individual services, and the specific boundaries that separate internal logic from external integrations.

5.1.1 Architectural Style

The development of the application was based on Service-Oriented Architecture (SOA). SOA is an architectural style in which a system is built as a collection of modular, reusable, and network-accessible services. Each service represents a distinct business capability — such as user management, bank integration, data synchronization, or reporting — and exposes its functionality through standardized interfaces, typically REST endpoints.

Although SOA and microservices share conceptual similarities — such as modularization, service boundaries, and network-based communication — they differ significantly in granularity, autonomy, communication style, governance, and deployment. In SOA, services can be independent, but they often share centralized components, such as databases, message busses, or identity providers. In microservices, each service is fully autonomous, owning its own database, technology stack, and deployment pipeline. This eliminates shared dependencies but increases operational complexity.

For this personal finance application, SOA provides an optimal balance. It allows for logical separation of domains without the heavy operational overhead of managing distributed databases and complex deployment pipelines required by pure microservices. As the system evolves and the need for higher scalability or independent deployment grows, this SOA foundation can naturally be refined into a microservices architecture.

5.1.2 Architecture Diagram

The architecture diagram illustrates the high-level structure of the application, showing the interaction between the frontend, backend services, external banking APIs, and the database.

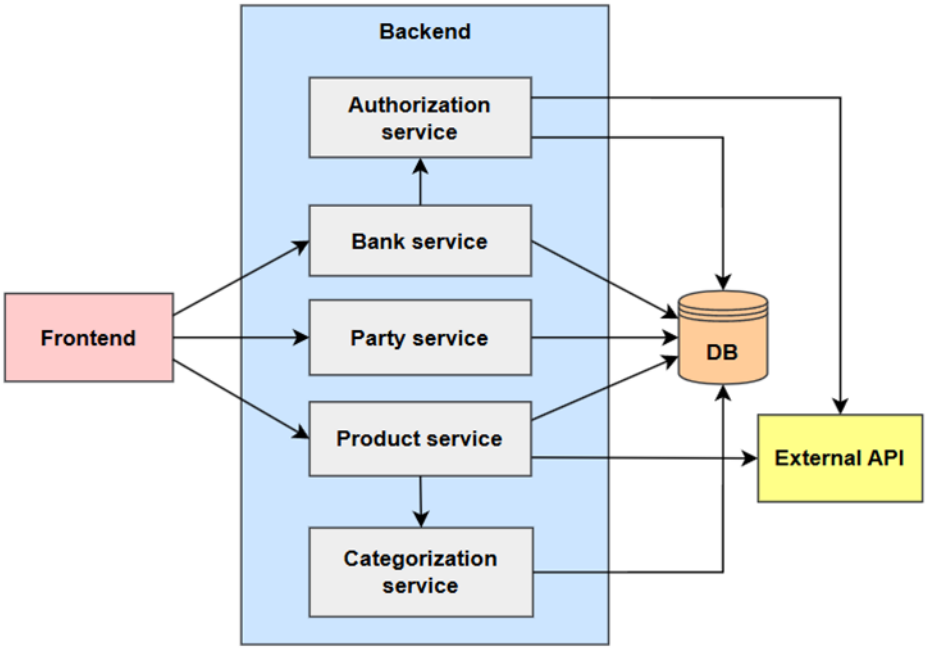

Figure 5.1. Application Architecture

The diagram in Figure 5.1 illustrates the Service-Oriented Architecture adopted for the system. The Frontend acts as the client layer, initiating requests to the backend. The Backend is decomposed into five distinct functional services: the Authorization service, the Bank service, the Party service, the Product service, and the Categorization service. Unlike a pure microservices architecture, where each service strictly owns its own database, these services share a central Database for persistence, ensuring data consistency while keeping the infrastructure manageable. The Product service also acts as a gateway to External APIs, handling the retrieval of banking data.

5.1.3 Internal Architecture

To ensure separation of concerns and maintainability, each service in the system follows a strict layered architecture composed of three main layers:

1. **Presentation Layer** — this layer exposes REST endpoints that allow clients to interact with the system. It handles incoming HTTP requests, validates input structures, and delegates execution to the domain layer. The presentation layer is strictly stateless and does not contain any business logic.
2. **Domain Layer** — this layer encapsulates the core business logic and workflows. It orchestrates operations between repositories, external providers, and domain objects. It is responsible for authorization checks, data transformation, validation rules, and transaction handling. It also contains domain entities and value objects representing the system's primary business concepts.
3. **Messaging and Persistence Layer** — this layer forms the boundary between the internal business logic and external infrastructure. The Messaging component acts as an adapter for external API implementations. Since each bank may provide different endpoints, data formats, authentication flows, or rate limits, it is essential to centralize and standardize these interactions here.

In addition to external messaging, this bottom layer manages data persistence using JPA and Hibernate. It abstracts database operations and provides a clean interface for

the Domain Layer to retrieve and store domain entities without needing to understand the underlying SQL dialects or table structures.

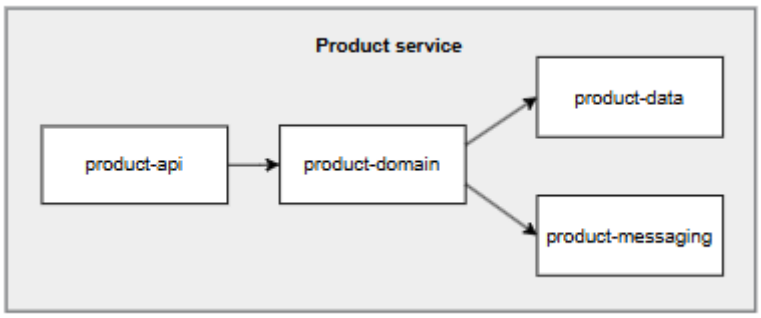

Figure 5.2. Internal Architecture

The internal architecture of the backend services, exemplified here in Figure 5.2 by the Product service, follows this strict layered pattern. The product-api component serves as the Presentation Layer, exposing REST endpoints to the client. This layer passes requests to the product-domain, which encapsulates the business logic and domain entities. The domain layer interacts with two downstream components: product-data, which handles persistence via the database, and product-messaging, which manages integrations with external banking systems.

5.1.4 Component Diagram

The component diagram illustrates the internal structure of the backend system and its interaction with the frontend, external bank APIs, and the database. It highlights how the conceptual layers described above are realized as actual software components.

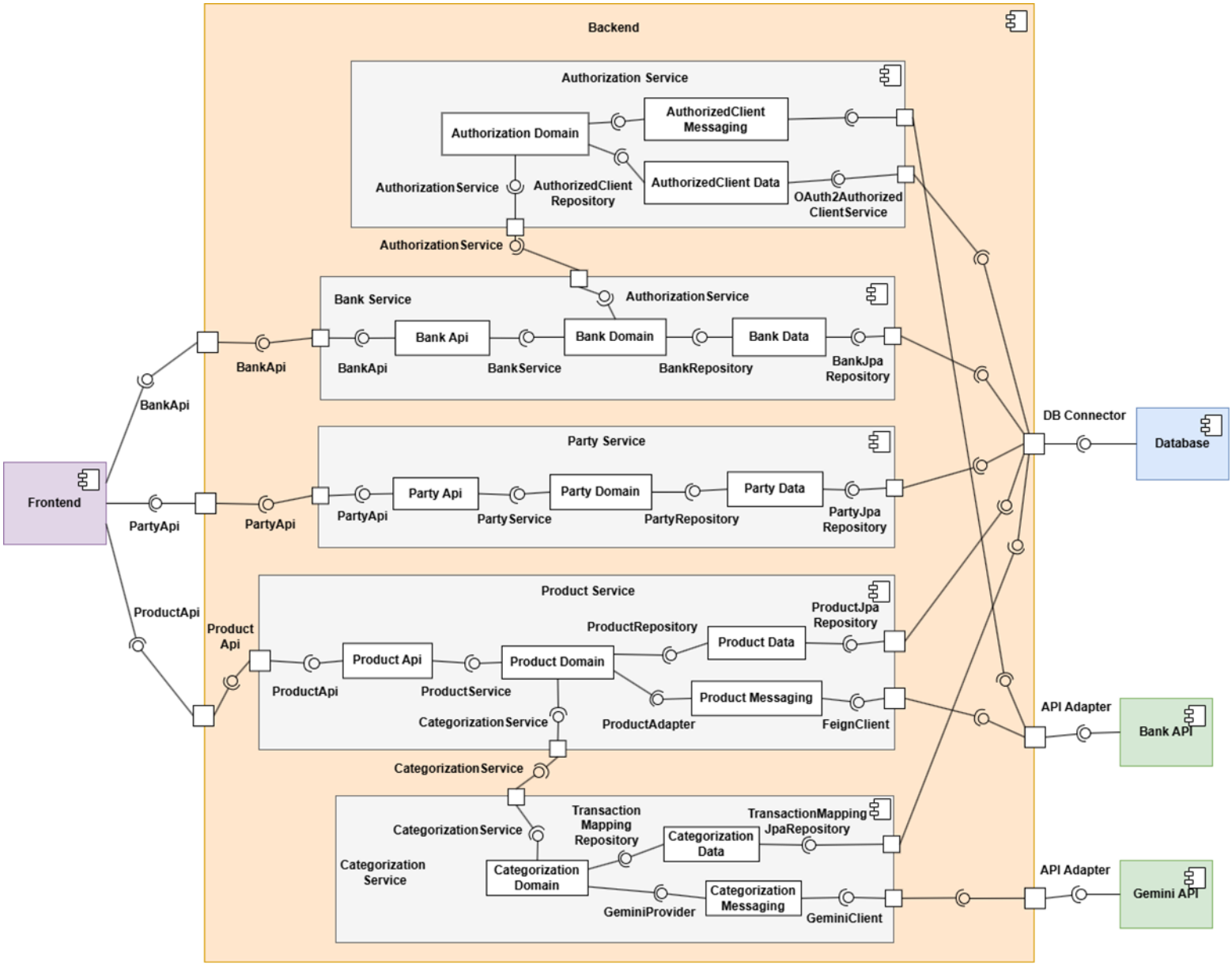

Figure 5.3. Component Diagram

The frontend communicates with the backend through exposed REST APIs. Each request is routed to the appropriate service via dedicated controllers, which represent the entry point to the backend. The diagram shows the four main backend services: Authorization Service, Bank Service, Profile Service, and Product Service. Each service is isolated within its own boundary, emphasizing modularity and preventing business logic from bleeding across domains.

Within each service, the standard layered architecture is applied. The controller layer handles incoming HTTP requests and forwards them to the application API layer. The API layer defines service-specific interfaces and orchestrates calls to the domain layer.

Persistence responsibilities are delegated to repository and data components (Authorization Data, Bank Data, Party Data, Product Data), which manage the physical database transactions. Communication with external banking systems is handled exclusively through adapter components. These APIs adapters provide a standardized

interface for interacting with varying bank APIs, effectively shielding internal services from external changes, authentication complexities, and network failures.

Overall, the component diagram demonstrates a clean separation between the presentation, application, domain, and infrastructure layers. It highlights the loose coupling between services, clear responsibility boundaries, and the high extensibility of the system.

To provide further context on the system's modularity, the core business responsibilities delegated to each backend service are defined as follows:

- **Authorization Service** — acts as the dedicated security backbone and token vault for all external banking integrations. It centrally manages OAuth2 client registrations, secure token acquisition, and the entire cryptographic lifecycle. By handling the storage, retrieval, and automated refreshing of access and refresh tokens, it ensures that sensitive credentials are strictly isolated from the rest of the application's business logic.
- **Party Service** — functions as the centralized user lifecycle manager. It completely isolates user identity and profile management from the financial domains.
- **Bank Service** — encapsulates the complexity of the Open Banking protocols and manages the logical state of connected banking institutions. It abstracts the highly variable integration details of different Czech banks into a unified internal interface, tracking user consent scopes and connection statuses (such as initiated, active, or revoked). Crucially, this service does not directly handle or store any security tokens. Instead, it delegates all token retrieval and refresh operations to the Authorization Service, allowing it to focus purely on orchestrating external bank communication and metadata.
- **Product Service** — serves as the core financial engine and data aggregation layer of the application. It is responsible for orchestrating financial data retrieval, normalizing data structures from various external banks into a single domain model, and applying business rules such as transaction categorization. Furthermore, it merges synchronized API data with manually created products and evaluates real-time spending against user-defined budgets.

5.2 Data design

The data design defines how information is structured, stored, and processed within the application. It focuses on the consistency, integrity, and normalization of financial data retrieved from external APIs, as well as manually created fictive data. The design ensures that real and fictive accounts can coexist within a unified data model, providing a seamless experience for the user, regardless of the data source.

5.2.1 Database Model

The database model describes the relational structure used to persist application data, user profiles, and external integration metadata. The model is designed to support both automated financial data integrations and manual fictitious data. For clarity and modularity, the overall database architecture is divided into the following specialized schemas.

5.2.1.1 Party Schema

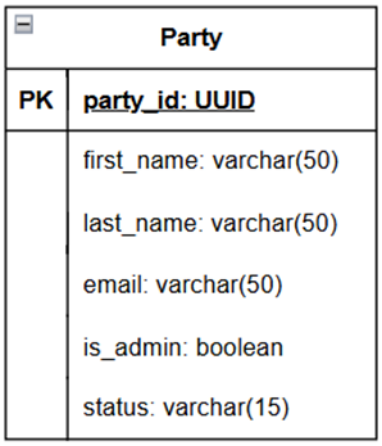

Figure 5.4. Party Schema

As illustrated in Figure 5.4, the central `Party` entity represents the system's users. This schema is responsible for storing their profile information, contact details, and administrative status, acting as the foundational root that links all other user-specific records in the application.

5.2.1.2 Authorized Client Schema

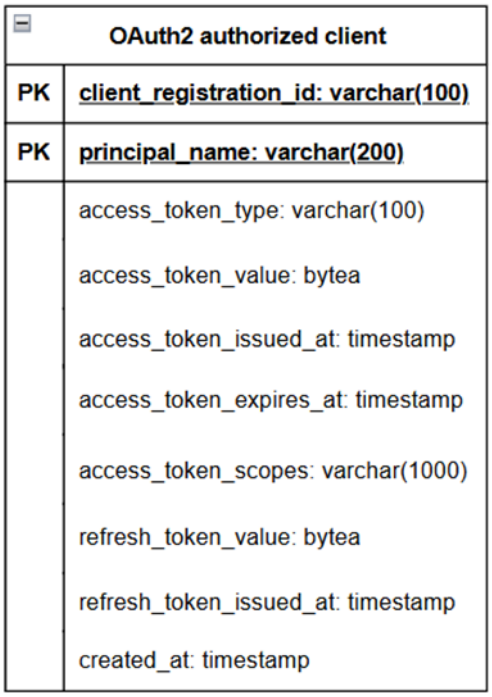

Figure 5.5. Authorized Client Schema

To handle secure communication with external banking APIs, the schema leverages Spring Security's built-in OAuth2 capabilities, depicted in Figure 5.5. The `OAuth2 authorized client` table relies on a standardized schema utilized by Spring's `OAuth2AuthorizedClientService`. Its primary responsibility is to securely persist authorized client instances across application restarts. This includes storing the

`access_token_value`, `refresh_token_value`, their respective expiration timestamps, and granted scopes. By delegating this to Spring Security, the application ensures that sensitive token lifecycle management (such as automatic token refreshing) is handled safely.

5.2.1.3 Bank Schema

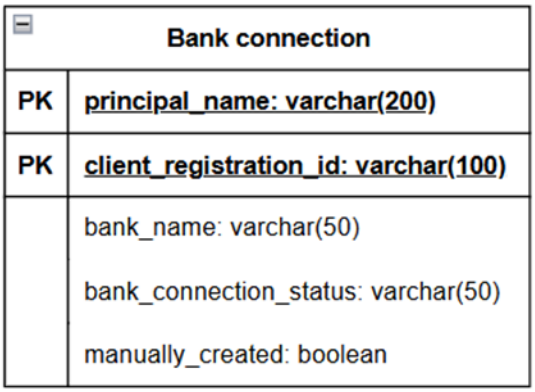

Figure 5.6. Bank Schema

`Bank connection` table, detailed in Figure 5.6, stores application-specific data about a user's linked bank. It utilizes a composite primary key — `principal_name` (representing the user) and `client_registration_id` (representing the specific bank provider). It tracks the `bank_name`, the current `bank_connection_status`, and a `manually_created` flag.

5.2.1.4 Product Schema

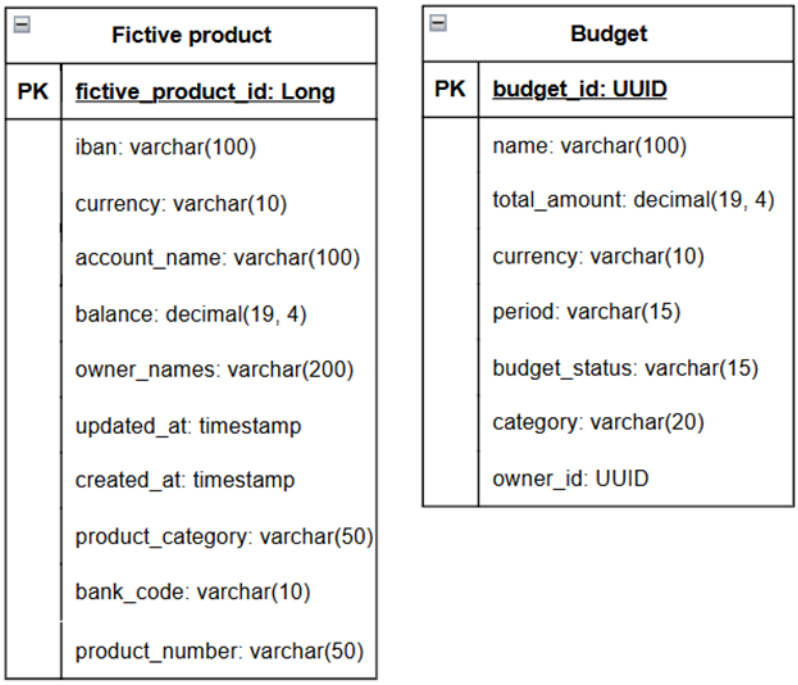

Figure 5.7. Product Schema

The product schema, presented in Figure 5.7, manages the financial accounts and balances within the system. For manual tracking, the schema uses the `Fictive product`

table. This table allows users to define custom, manually managed accounts, storing necessary details such as IBAN, currency, balance, and bank code in a single consolidated entity.

5.2.1.5 Categorization Schema

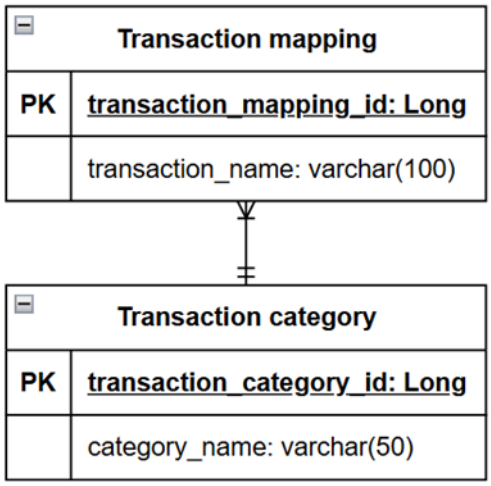

Figure 5.8. Categorization Schema

The categorization schema, illustrated in Figure 5.8, is utilized by the categorization service to store and manage how raw bank transactions are classified.

The `Transaction category` table defines the standardized categories available within the system. It contains a primary key, `transaction_category_id`, and the `category_name` representing the classification (e.g., Groceries, Salary).

The `Transaction mapping` table is responsible for linking specific raw transaction information encountered in banking data (stored in the `transaction_name` field) to these standardized categories. It uses `transaction_mapping_id` as its primary key. The schema establishes a many-to-one relationship between `Transaction mapping` and `Transaction category`, allowing multiple distinct transaction names from various bank records to be cleanly grouped under a single unified category.

5.2.2 Domain Class Model

The domain class model represents the core business entities and their relationships within the application. It abstracts technical concerns and focuses on financial concepts such as users, bank connections, accounts, and transactions. This model serves as the foundation for business logic implementation and aligns with the layered architecture used in the system.

5.2.2.1 Party Service

This section outlines the object-oriented representation of user profiles within the system. The model isolates the properties required to identify, authenticate, and manage the privileges of individuals using the platform.

The Party Service domain model focuses on user management. The primary class, `UserDomainEntity`, contains personal attributes such as first name, last name, and email. It also manages access control via the `isAdmin` flag and the status attribute. The status is defined by the `UserStatus` enumeration, which restricts users to either an ACTIVE or DEACTIVATED state.

5.2.2.2 Bank Service

The class model for the Bank Service maps the connection metadata required to manage integrations with third-party banking institutions, as well as custom banks created by the user. It abstracts the integration details and connection lifecycle states into manageable domain entities, delegating actual token management to the Authorization Service.

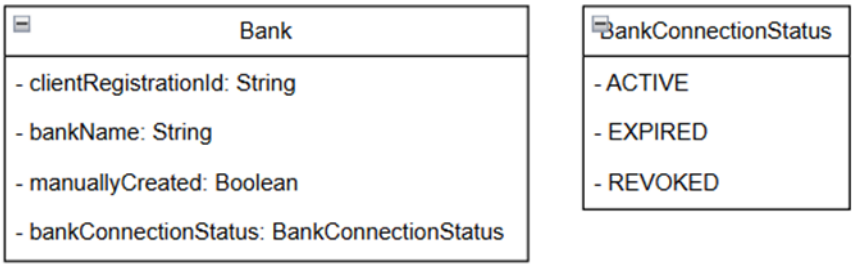

Figure 5.9. Domain Class Model - Bank Service

This domain model handles the metadata and state of banking integrations rather than the security credentials. The primary entity, `Bank`, identifies the institution using a `clientRegistrationId` and records the `bankName`. It also utilizes a `manuallyCreated` flag to seamlessly distinguish between automated integrations and user-defined banks. The lifecycle of the integration is managed by the `bankConnectionStatus` attribute, utilizing the `BankConnectionStatus` enum to track if a connection is currently ACTIVE, EXPIRED, or REVOKED.

5.2.2.3 Product Service

The Product Service domain model encompasses the most data-intensive part of the application. It defines the hierarchical and relational bindings between financial products (accounts), individual transactions, data aggregations, and user-defined budgets.

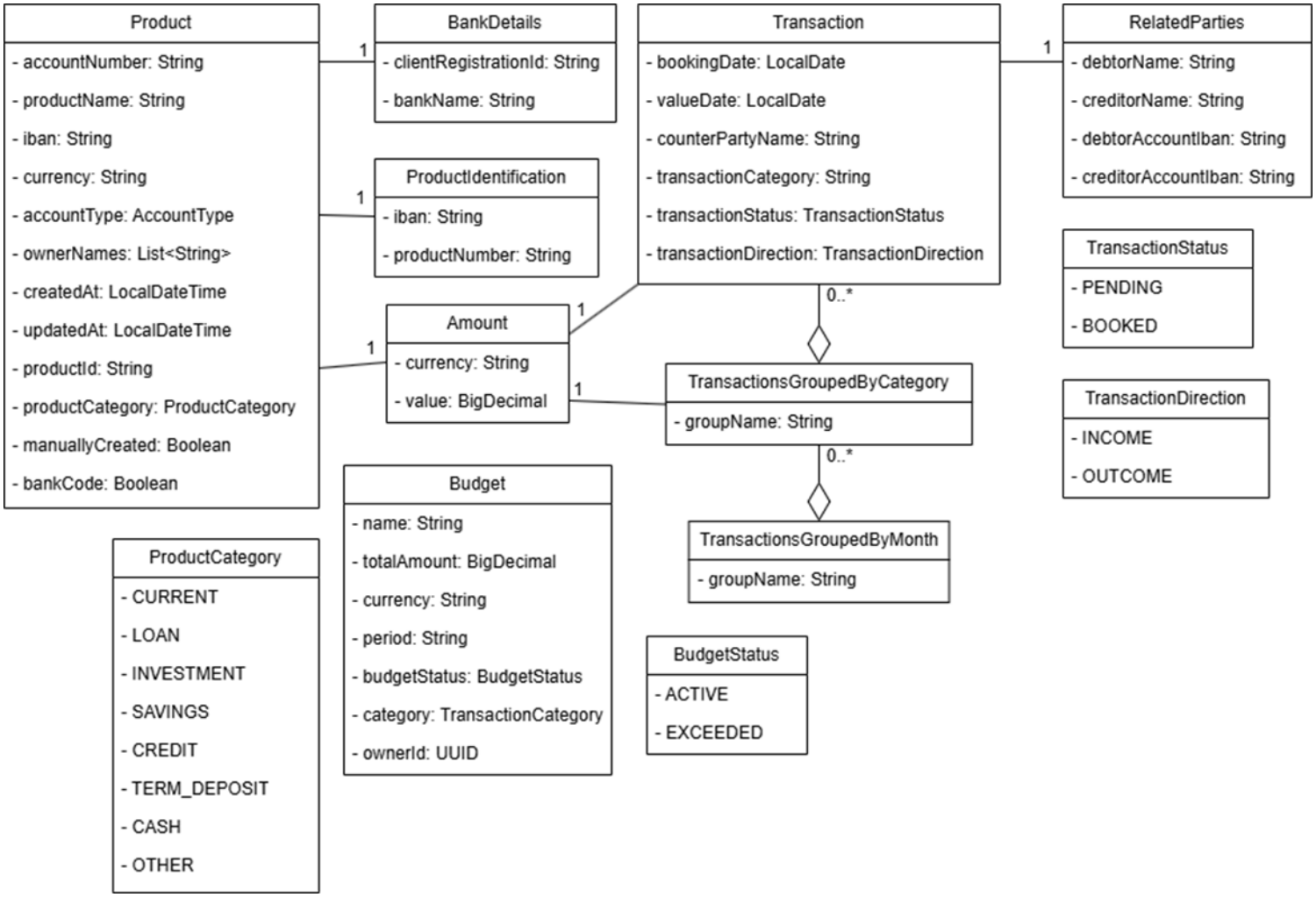

Figure 5.10. Domain Class Model - Product Service

The Product Service model forms the core financial logic of the system. It is centered around the `Product` entity, which serves as a unified representation for both synchronized bank accounts and manually created fictive accounts (distinguished by the `manuallyCreated` flag). To maintain clean data encapsulation, `Product` delegates specific attributes to associated entities like `BankDetails`, `ProductIdentification`, and `Amount`.

Individual financial movements are recorded by the `Transaction` entity. This class captures core details such as booking dates, processing states (`TransactionStatus`), and cash flow direction (`TransactionDirection`). Complex counterpart details (debtor and creditor information) are cleanly extracted into a dedicated `RelatedParties` entity.

To directly support frontend analytics and dashboard visualizations, the model introduces specialized composition structures: `TransactionsGroupedByCategory` and `TransactionsGroupedByMonth`, which hierarchically organize raw transactions for efficient reporting.

Finally, the `Budget` entity enables goal-oriented financial planning, allowing users to set monetary targets based on a specific `TransactionCategory` and track progress via the `BudgetStatus`.

5.2.2.4 Categorization Service

The Categorization Service is responsible for the automated classification of incoming bank transactions. To achieve this, the service processes each transaction by concatenating its counterparty name and transaction direction (INCOME or OUTCOME) to form a unified `transaction_name`.

Figure 5.11. Domain Class Model - Categorization Service

The categorization workflow operates in a two-tier mechanism to balance performance and accuracy:

Mapping Resolution: The service first queries the `Transaction mapping` table to determine if the generated `transaction_name` has been previously classified. If an existing mapping is found, the service immediately applies the associated `TransactionCategory`.

AI-Driven Categorization: If no existing mapping is found in the database, the service falls back to an AI-driven approach. It integrates with the Gemini API provider, supplying the unrecognized `transaction_name` and prompting the Gemini AI Model to select the most appropriate classification strictly from the predefined `TransactionCategory` enumeration.

Once the Gemini API returns a valid category, the service applies this classification to the current transaction. Furthermore, it immediately persists this newly discovered relationship into the `Transaction mapping` table. This self-learning caching mechanism ensures that the system continuously expands its internal mapping dictionary, meaning future transactions with the exact same name will be categorized instantly via the database without requiring repeated, latency-heavy calls to the external AI provider.

5.3 External API Overview

Modern personal finance applications rely heavily on secure integrations with banking systems. In the Czech Republic, most major banks expose standardized PSD2 interfaces

that allow licensed third-party providers to access account information with explicit user consent. These APIs follow the Czech Open Banking Standard or the Open Banking UK model, offering consistent structures for account, balance, and transaction retrieval. This chapter provides an overview of the external APIs relevant to the application, including their capabilities, requirements, and access conditions for both sandbox and production environments.

5.3.1 Conclusion

The analysis of external banking APIs available in the Czech Republic demonstrates that, while most major banks offer comprehensive PSD2-compliant Account Information Services, access to real production data is strictly regulated and limited to licensed third-party providers. The sandbox environments offered by these banks significantly lower the access barriers by removing the need for PSD2 licenses and certificates, making them well suited for development, prototyping, and functional demonstrations.

Chapter 6
Conclusion

This project demonstrates the design of a secure and user-friendly web application for managing personal bank accounts. Through analysis of existing finance platforms and Czech banking APIs, a solution was proposed that aggregates financial data, supports budgeting and visualization, and respects PSD2 and GDPR requirements. The chosen service-oriented architecture and technology stack ensure scalability, maintainability, and resilience. Although production access to most bank APIs is limited by regulatory constraints, the application remains functional through sandbox integrations and fictive accounts. The project provides a solid foundation for further development, including potential commercial deployment or extension toward advanced financial analytics.

References

[1] Vít Zýka. *FELthesis – LaTeX templates for thesis on CTU FEL*. 2012. `http://zyka.net/felthesis/felthesis.zip`.

[2] *Grafický manuál identity Českého vysokého učení technického v Praze*. Kristina Fišerová. `https://www.cvut.cz/logo-a-graficky-manual`.

Appendix A
Zadání práce

Appendix B
Zkratky a symboly

Tento text je až na výjimky převzat z [1].

B.1 Zkratky

Jako příklad pro popis zkratek poslouží pojmy ze světa TEXu.

TEX — Program na přípravu elektronické sazby vysoké kvality vytvořený Donaldem Knuthem. Program zahrnuje interpret makrojazyka. Název programu se vyslovuje „tech“.

LuaTEX — Rozšířený program TEX o možnost užití Unicode fontů a programovacího jazyka Lua.

METAFONT — Program a makro jazyk pro generování fontů z vektorového do bitmapového formátu vytvořený Donaldem Knuthem.

METAPOST — Program generující vektorovou grafiku založený na METAFONTu vytvořený Johnem Hobby.

plainTEX — Originální TEXový formát (rošíření na úrovni makrojazyka). Je součástí každé distribuce TEXu a je vytvořen Donaldem Knuthem.

OpTEX — TEXový formát rozšiřující plainTEX s využitím maker OPmac pro LuaTEX.

$\mathcal{CS}$plain — TEXový formát rozšiřující plainTEX o možnosti sazby v českém a slovenském jazyce vytvořený Petrem Olšákem.

LATEX — Nejznámější TEXový formát (rozšíření na úrovni makrojazyka) vytvořený Leslie Lamportem. Existuje obludné množství různých balíčků, které pomocí makrojazyka TEXu dále rozšiřují výchozí možnosti LATEXu. Rozličné uživatelské požadavky jsou nejčastěji řešeny použitím vhodného balíčku.

OPmac — Olšákova PlainTEXová makra nabízející uživatelům plainTEXu podobné možnosti, jako LATEX, ovšem přímočařeji a jednodušeji.

ConTEXt — Typografický systém vystavěný na LuaTEXu a na předpřipravených makro souborech vytvořený týmem v čele s Hansem Hagenem. Rozličné uživatelské požadavky jsou nastavovány pomocí přiřazení hodnot klíčovým slovům společně s možností TEXového, METAPOSTího a Lua programování.

B.2 Symboly

π — Konečná verze TEXu zmíněna v Knuthově TEXtamentu.

e — Konečná verze METAFONTu.

2ε — Současná verze LATEXu používaná od roku 1994. Počítá se s ní jako s přechodnou verzí mezi původní Lamportovou verzí LATEX 2.09 a cílovou verzí LATEX 3. Tento přechodný stav už trvá 27 let.

B.3 Soubory, které jsou součástí CTUstyle3

`ctustyle3.tex`	TEXová makra implementující šablonu ve verzi 3 (pro OpTEX).
	Logo ČVUT podle[2] v modré a černé variantě.
`Technika-Regular.otf`	Metrika a kresby písma **Technika-Regular**.
`Technika-Italic.otf`	Metrika a kresby písma ***Technika-Italic***.
`Technika-Book.otf`	Metrika a kresby písma Technika-Book.
`Technika-BookItalic.otf`	Metrika a kresby písma *Technika-BookItalic*.
`Technika-Bold.otf`	Metrika a kresby písma **Technika-Bold**.
`Technika-BoldItalic.otf`	Metrika a kresby písma ***Technika-BoldItalic***.
`ctustyle-doc.tex`	Hlavní zdrojový soubor tohoto dokumentu.
`uvod.tex, popis.tex, prilohy.tex`	Zdrojové soubory čtené z `ctustyle-doc.tex` s jednotlivými kapitolami tohoto dokumentu.
`cmelak1.jpg`	Obrázek použitý v ukázce, jak vložit obrázek.
`mybase.bib`	Údaje použité pro generování seznamu literatury.
`ctustyle-doc.pdf`	Tento dokument.
`slides.tex`	Zdrojový text ilustrující použití CTUslides.
`slides.pdf`	Výstupní ukázka ilustrující použití CTUslides včetně návodu k použití.